



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема роботи: Використання шаблону проєктування **Bridge (Міст)** у програмному забезпеченні.

Мета роботи: Ознайомитися з шаблоном проєктування **Bridge**, його призначенням та структурою, а також реалізувати даний шаблон у програмному проєкті з метою відокремлення абстракції від реалізації та забезпечення їх незалежного розвитку.

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	2
Завдання:	2
Діаграма класів:	3
Опис реалізації:	3
Вихідний код	5
Контрольні запитання.....	9
Висновки	12

Теоретичні відомості

Шаблон проєктування Bridge

Bridge (Міст) – це структурний шаблон проєктування, який дозволяє відокремити абстракцію від її реалізації таким чином, щоб обидві могли змінюватися незалежно одна від одної.

Основна ідея шаблону полягає у створенні двох незалежних ієрархій:

- ієрархії **абстракцій** (високорівнева логіка);
- ієрархії **реалізацій** (низькорівнева функціональність).

Абстракція містить посилання на об'єкт реалізації та делегує йому виконання частини роботи.

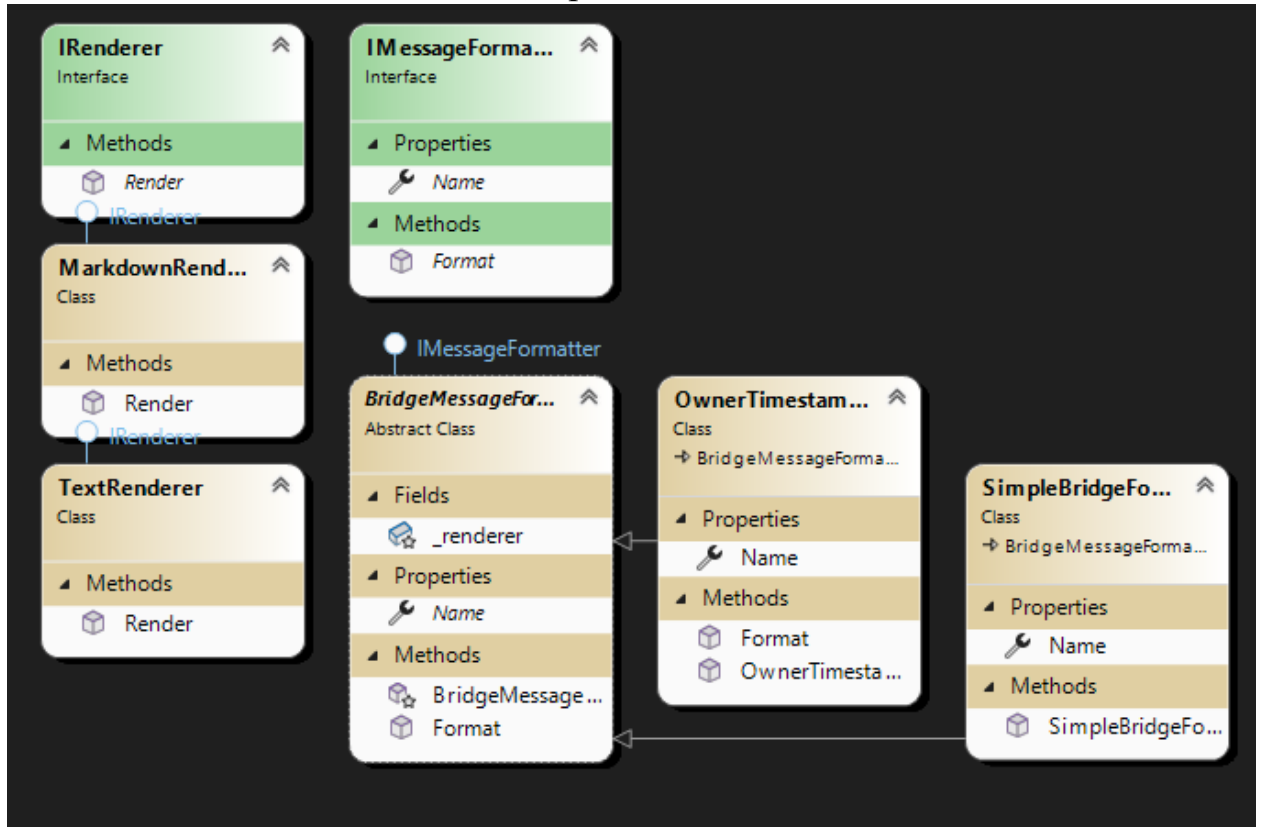
Хід виконання:

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів:



Опис реалізації:

Опис реалізації в лабораторній роботі

У даній лабораторній роботі шаблон Bridge було реалізовано для задачі форматування та рендерингу тексту повідомлень у чаті.

Метою реалізації є розділення:

- політики форматування повідомлення (що і як відображати);
- механізму рендерингу тексту (яким способом формується фінальний рядок).

Сторона реалізації (Implementation)

Було створено інтерфейс:

`IRenderer`

який визначає низькорівневий механізм рендерингу повідомлення. Конкретні реалізації:

- `TextRenderer` – рендеринг простого тексту;
- `MarkdownRenderer` – рендеринг повідомлення у форматі Markdown.

Дані класи відповідають ролі `ConcreteImplementor` шаблону Bridge.

Сторона абстракції (Abstraction)

Для представлення високорівневої логіки форматування було створено абстрактний клас:

`BridgeMessageFormatter`

Даний клас:

- реалізує інтерфейс `IMessageFormatter`;
- зберігає посилання на `IRenderer`;
- делегує низькорівневе формування тексту об'єкту реалізації.

Розширені абстракції (Refined Abstraction)

Було реалізовано кілька конкретних форматерів:

- `SimpleBridgeFormatter` – базове форматування без додаткової логіки;
- `OwnerTimestampBridgeFormatter` – форматування з додаванням імені користувача та часу відправлення.

Ці класи відповідають ролі `RefinedAbstraction` та реалізують різні політики представлення повідомлення.

Клієнтський код

Клієнтський код (`RepositoryMessages`) працює з інтерфейсом `IMessageFormatter` та не має інформації про конкретні реалізації рендерингу або форматування. Це забезпечує слабку зв'язаність та прозорість використання шаблону Bridge.

Відповідність шаблону Bridge

Реалізація відповідає шаблону Bridge з таких причин:

1. Наявні дві незалежні ієрархії – форматери та рендерери.
2. Абстракція містить реалізацію – BridgeMessageFormatter зберігає IRenderer.
3. Можливість незалежного розширення – додавання нового формatera або рендерера не потребує змін у іншій ієрархії.
4. Відсутність залежності клієнта від реалізацій – клієнт працює лише з абстракціями.

Переваги реалізації

- зменшено зв'язаність між логікою форматування та рендерингу;
- спрощено розширення системи новими способами відображення повідомлень;
- покращено читабельність та підтримуваність коду;
- забезпечено відповідність принципам SOLID.

Можливі напрямки розширення

Реалізація шаблону Bridge може бути розширена шляхом:

- додавання нових рендерерів (HTML, RichText, локалізовані рендерери);
- створення нових форматерів з додатковою бізнес-логікою;
- поєднання Bridge з іншими шаблонами (Strategy, Abstract Factory) для динамічного вибору комбінацій форматування.

Вихідний код

```
// Formatting/Bridge/BridgeMessageFormatter.cs
```

```
using Skype.Data;
```

```
namespace Skype.Formatting.Bridge
```

```

{
    using Skype.Formatting;

    // Abstraction side of the Bridge: base formatter holding a renderer.
    public abstract class BridgeMessageFormatter : IMessageFormatter
    {
        protected readonly IRenderer _renderer;

        protected BridgeMessageFormatter(IRenderer renderer)
        {
            _renderer = renderer;
        }

        public abstract string Name { get; }

        // High-level formatting uses the renderer (implementation) to produce final string.
        public virtual string Format(Message message)
        {
            return _renderer.Render(message);
        }
    }
}

```

```

// Formatting/Bridge/IRenderer.cs
namespace Skype.Formatting.Bridge
{
    using Skype.Data;

    // Implementation side of the Bridge: low-level rendering engines.
    public interface IRenderer
    {
        string Render(Message message);
    }
}

```

```
// Formatting/Bridge/MarkdownRenderer.cs

using Skype.Data;
using System.Net;

namespace Skype.Formatting.Bridge
{
    // Markdown renderer (low-level implementation).
    public class MarkdownRenderer : IRenderer
    {
        public string Render(Message message)
        {
            // Minimal example: escape and wrap as markdown
            var text = WebUtility.HtmlEncode(message.Text);
            return $"**{message.OwnerId}**\n\n{text}";
        }
    }
}
```

```
// Formatting/Bridge/OwnerTimestampBridgeFormatter.cs

using Skype.Data;

namespace Skype.Formatting.Bridge
{
    // Concrete abstraction: adds owner/time metadata then delegates rendering of body.
    public class OwnerTimestampBridgeFormatter : BridgeMessageFormatter
    {
        public override string Name => "bridge-owner-ts";

        public OwnerTimestampBridgeFormatter(IRenderer renderer) : base(renderer) { }

        public override string Format(Message message)
        {
            var body = _renderer.Render(message);
```

```

        return $"[{message.TimeSend:yyyy-MM-dd HH:mm}] {body}";
    }
}
}

```

// Formatting/Bridge/SimpleBridgeFormatter.cs

using Skype.Data;

namespace Skype.Formatting.Bridge

```

{
    // Concrete abstraction: simple pass-through to renderer.
    public class SimpleBridgeFormatter : BridgeMessageFormatter
    {
        public override string Name => "bridge-simple";

        public SimpleBridgeFormatter(IRenderer renderer) : base(renderer) { }

        // Inherits Format -> renderer.Render(message)
    }
}

```

// Formatting/Bridge/TextRenderer.cs

using Skype.Data;

using System.Net;

namespace Skype.Formatting.Bridge

```

{
    // Simple plain-text renderer (low-level implementation).
    public class TextRenderer : IRenderer
    {
        public string Render(Message message)
        {
            // Escape text minimally and return full text body
            var text = WebUtility.HtmlEncode(message.Text);

```



```

        return $" {text}";
    }
}
}

```

Контрольні запитання

1. Яке призначення шаблону «Посередник»?

Шаблон **Mediator (Посередник)** призначений для централізації взаємодії між об'єктами, зменшуючи кількість прямих зв'язків між ними та знижуючи загальну зв'язаність системи.

2. Структура шаблону «Посередник»

```

ColleagueA ----\
                \
ColleagueB -----> Mediator
                /
ColleagueC ----/

```

3. Класи шаблону «Посередник» та їх взаємодія

- **Mediator** – інтерфейс або абстрактний клас посередника.
- **ConcreteMediator** – реалізує логіку взаємодії між об'єктами.
- **Colleague** – об'єкти, які взаємодіють через Mediator.
- **ConcreteColleague** – конкретні реалізації учасників.

Colleague не взаємодіють напряду, а повідомляють Mediator про події.

4. Яке призначення шаблону «Фасад»?

Шаблон **Facade (Фасад)** надає спрощений інтерфейс до складної підсистеми, приховуючи її внутрішню структуру від клієнта.

5. Структура шаблону «Фасад»

Client

|

v

Facade

|

v

SubsystemA SubsystemB SubsystemC

6. Класи шаблону «Фасад» та їх взаємодія

- **Facade** – спрощений інтерфейс.
- **Subsystem classes** – класи підсистеми з повною функціональністю.
- **Client** – використовує лише Facade.

Клієнт не має прямого доступу до підсистеми.

7. Яке призначення шаблону «Міст»?

Шаблон **Bridge (Міст)** призначений для відокремлення абстракції від її реалізації таким чином, щоб вони могли змінюватися незалежно одна від одної.

8. Структура шаблону «Міст»

Abstraction

|

v

Implementor (interface)

^

|

ConcreteImplementorA ConcreteImplementorB

9. Класи шаблону «Міст» та їх взаємодія

- **Abstraction** – високорівнева логіка.

- **RefinedAbstraction** – розширення абстракції.
- **Implementor** – інтерфейс реалізації.
- **ConcreteImplementor** – конкретні реалізації.

Abstraction делегує роботу Implementor.

10. Яке призначення шаблону «Шаблонний метод»?

Шаблон **Template Method (Шаблонний метод)** визначає загальний алгоритм у базовому класі та дозволяє підкласам перевизначати окремі кроки алгоритму без зміни його структури.

11. Структура шаблону «Шаблонний метод»

AbstractClass

```
|
|
+--> templateMethod()
|   |
|   +--> step1()
|   +--> step2()
|
```

ConcreteClass

12. Класи шаблону «Шаблонний метод» та їх взаємодія

- **AbstractClass** – містить шаблонний метод і абстрактні кроки.
- **ConcreteClass** – реалізує або перевизначає кроки алгоритму.

Послідовність кроків фіксована в AbstractClass.

13. Чим відрізняється «Шаблонний метод» від «Фабричного методу»?

- **Template Method** визначає алгоритм.
- **Factory Method** визначає створення об'єкта.
- Template Method керує послідовністю дій.
- Factory Method інкапсулює вибір конкретного продукту.

14. Яку функціональність додає шаблон «Міст»?

Шаблон Bridge додає можливість:

- незалежно розширювати абстракцію та реалізацію;
- комбінувати різні реалізації з різними абстракціями;
- зменшити кількість класів у порівнянні з наслідуванням.

Висновки

У ході виконання лабораторної роботи було реалізовано шаблон проєктування **Bridge**, який дозволив відокремити високорівневу логіку форматування повідомлень від низькорівневої реалізації рендерингу. Застосування шаблону забезпечило гнучкість, розширюваність та чистоту архітектури програмного забезпечення, а також дозволило незалежно розвивати обидві ієрархії компонентів.